

课程笔记：Anthropic Workshop：如何构建能运行数小时的 Agent

AI Engineer / Anthropic Applied AI; SSS & Codex 整理

2026-07-05



视频作者/频道：AI Engineer

发布日期：2026-05-18

视频时长：01:15:40

视频链接：<https://www.youtube.com/watch?v=mR-WAvEPRwE>

PDF 制作 Skill: <https://github.com/wdkns/wdkns-skills>

目录

1 为什么长时间运行的智能体会失控

来源范围：本章对应视频 00:01:21--00:05:58。核心结论是：长时间运行 agent 的失控，先要从 context、planning、judgment 三个失败来源理解；修复路径也有两条，一条进入模型权重，一条进入模型外部的 harness（智能体运行框架）。

本章结论

长时间运行能力不是单一模型指标，而是一种工程系统属性。模型需要更强的内生能力，harness 也需要承担上下文管理、任务拆分、工具调用、状态持久化、权限控制和验收判断。只看一次漂亮 demo，无法解释为什么 agent 能从 20 分钟扩展到数小时甚至数天。

1.1 从 20 分钟到 days at a time：问题规模的变化

讲者用 Boris 对 Claude Code 一周年的回顾作为开场：一年前，Claude Code 还会在写 Bash 命令和转义字符串时挣扎，只能连续工作大约 20 分钟；一年后，Claude Code 自身的大部分代码已经由 Claude Code 参与编写，并且可以有效运行到 days at a time。这不是一个简单的“模型变聪明了”的故事，而是任务尺度发生了变化。

短任务像一次性答题：模型只要在当前上下文里理解问题、调用几个工具、给出答案。长任务更像多小时施工：需求会被拆开，状态会变化，工具结果会堆积，半成品需要验收，后续步骤还要记得前面已经做过什么。讲者在开场还强调，这类 workshop 关注的是 agents that can actually run for really long extended periods of time，例如 5 6 hour plus kind of runs。这里的关键不是“运行时间越长越好”，而是长时间运行暴露了短 demo 中被掩盖的系统性问题。

20 分钟与 days at a time 的差别

20 分钟 agent 仍然可以依赖一次会话里的短期记忆；数小时 agent 必须面对上下文膨胀、计划漂移、工具失败、状态恢复和质量验收。前者像临时便签，后者像一个要交接给下一班工程师的施工日志。

1.2 三类失败：Context、Planning、Judgment

讲者把长时间运行 agent 的失败先分成三类：context（上下文）、planning（计划）、judgment（判断）。这个划分很重要，因为很多看似“模型不够强”的现象，实际来自三种不同的工程约束。

第一类是 context。上下文可以理解为 agent 的工作笔记本：里面有用户要求、系统指令、工具输出、文件摘要、历史尝试和临时判断。短任务中，这本笔记本很薄，模型能快速翻完；长任务中，笔记本会越来越厚，旧信息会被新信息覆盖，工具输出会污染注意力，摘要会丢失细节。讲者称这种连贯性下降为 context rot。当模型接近上下文窗口末端时，还可能出现 ASR 记录为 context sense anxiety 的现象：模型意识到空间快用尽，于是急于收尾，提前把还没完成的工作说成完成。

第二类是 planning。长任务里的 planning 不只是列一个待办清单，而是像多小时施工排期：先决定地基，再决定管线，再决定验收顺序。如果计划太粗，模型可能一口气尝试完成所有内容，结果只做出半个功能；如果计划太细，前面一个错误会沿着后续步骤放大。讲者在本章只先指出基础问题：模型开箱即用往往不擅长规划，可能半途停止、忘记未完成项，或者上下文耗尽后留下一个半成品。

第三类是 judgment。讲者直接指出：models are really bad at judging their own output。这类失败更隐蔽，因为产物表面上看起来已经完成。例如，一个按钮出现在页面上，模型就认为功

能完成；实际后端不存在，点击也没有行为。这个问题类似让施工队自己盖楼、自己验收、自己签字：只要它对自己的半成品足够宽容，后续所有计划都会建立在错误完成状态上。

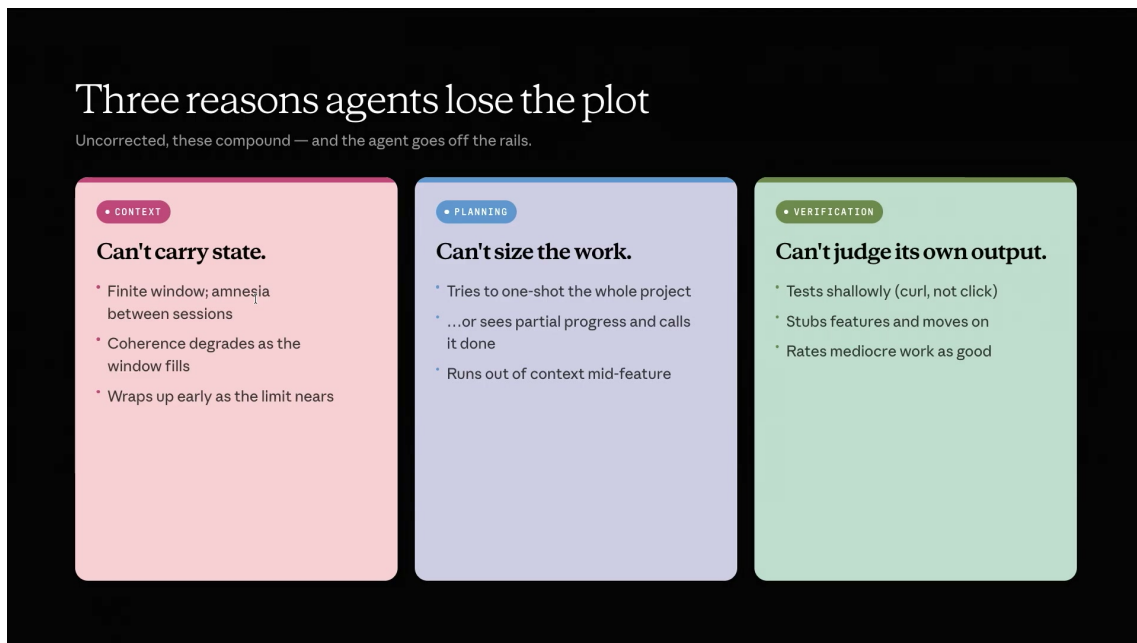


图 1: 长时运行 Agent 失控的三类来源：Context、Planning、Judgment。¹

不要把三类失败混成一个问题

Context 失败是“看不全或看不准工作笔记”；planning 失败是“不会把长任务排成可执行施工表”；judgment 失败是“验收员太容易接受半成品”。三者会互相放大，但修复手段不同。

1.3 两条修复路线：模型权重与 harness evolution

讲者随后给出两条修复路线。第一条是进入 **model weights (模型权重)**：通过训练让模型本身更擅长长期任务、上下文管理、工具选择、规划和判断。视频中提到的 meter chart 用一个“minimal scaffold (最小脚手架)”基准展示模型侧进步：在完成 50% 任务的口径下，从 Opus 3.7 的约 1 小时推进到 Opus 4.6 的 12 小时。

这里必须保留一个限制：这个 12 小时数字来自讲者所说的 minimal scaffold benchmark，不能写成所有 agent 都能稳定运行 12 小时。它说明模型能力边界在移动，不能替代具体任务里的验收、状态管理和失败恢复。

第二条路线是 **harness evolution (运行框架演化)**。harness 是模型外部的工程结构：核心 agent loop、工具调用、MCP server、sub-agent 委派、`claude.md`、skills、slash commands、权限系统，以及后续章节会展开的 evaluator、contract、trace reading 等机制。它像施工现场的脚手架、工单系统和质检流程：模型是工人和设计师，harness 决定它如何拿工具、如何记状态、如何交接、如何被验收。

¹ 视频画面时间区间：00:02:30–00:02:58。若为裁剪图，时间区间仍对应原视频画面。

Fix #1: Train the behavior into the weights.

Slow to ship — months per release. Free at runtime, works in every harness.

METR's task-completion time horizon: the human-time length of task a model completes with 50% reliability, in a minimal scaffold.

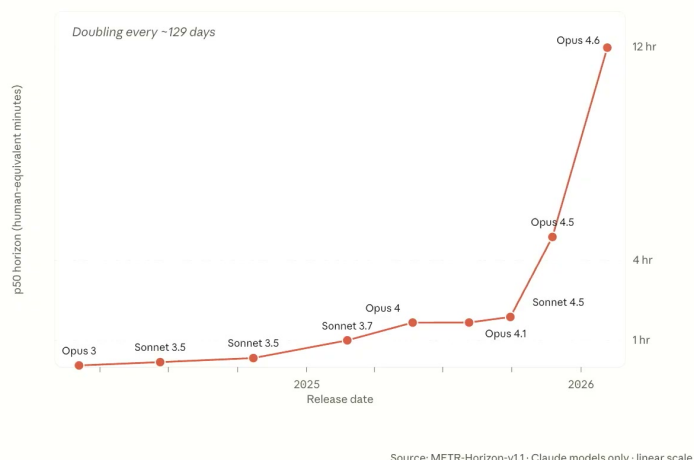


图 2: 模型侧进步可以延长 minimal scaffold 下的无人值守时间; 该图展示的是基准口径, 不是通用保证。²

讲者强调的关键关系是 co-evolution: 模型版本发布时, Anthropic 往往同时发布 harness primitives。原因很直接: 当模型某项能力变强, 旧 harness 中某些补丁会变得冗余; 当模型仍有缺口, harness 会把缺口外化成工具、状态、子代理、检查点或评估循环。长时间运行能力由这两条路线共同推进。

harness 的定义

本 PDF 中的 harness 指模型外部的智能体运行框架, 包括循环控制、工具接入、上下文策略、文件状态、权限、子代理、评估器和验收协议。它不是提示词装饰, 而是让模型工作可持续、可恢复、可检查的工程边界。

1.4 本章小结

本章的结论是: 长时间运行 agent 会失控, 根源通常落在 context、planning、judgment 三个桶里。Context 像会被污染和压缩的工作笔记, planning 像多小时施工排期, judgment 像容易给半成品盖章的验收员。模型权重可以提升内生能力, harness evolution 可以把状态、工具、验收和恢复机制工程化。后续章节讨论 Claude Code 的一年演化时, 应始终回到这三类失败: 每个新 primitive 都是在补某个具体缺口。

2 Claude Code 一年演化: 从工具到长时运行 primitives

来源范围: 本章对应视频 00:05:58--00:17:28。核心结论是: Claude Code 的一年演化展示了 harness 如何随模型能力改变; 早期主要依赖工具、验证和显式循环补足缺口, 后期逐步引入 checkpoints、Agent SDK、skills、sub-agents、Agent Teams、server-side compaction 和更大上下文, 把“能跑更久”拆成一组可复用 primitives。

² 视频画面时间区间: 00:04:16–00:04:43。若为裁剪图, 时间区间仍对应原视频画面。

本章结论

这一段历史不能写成产品发布时间流水账。每个节点都要回答它补了哪类失败：Computer Use 和 MCP 补工具与验证，Claude Code research preview 补真实开发反馈，Ralph loop 补可重复执行和 fresh context，checkpoints 补回退，skills 补上下文加载，sub-agents 和 Agent Teams 补并行分工，server-side compaction 和 1M context 改变上下文策略。

2.1 Prehistory: Sonnet 3.5、Computer Use 与 MCP

讲者把 Sonnet 3.5、Computer Use 与 MCP 称作 Claude Code 之前的 prehistory，因为它们先解决了 agent 作为工程执行者的两个前提：能看见结果，能使用工具。

Sonnet 3.5 的关键意义是 coding 能力开始显现，并且模型可以查看自己构建出的 artifact，再继续迭代。这里的 verification（验证）是长时运行的基础：如果 agent 无法观察产物，就只能靠文本想象判断是否完成。Computer Use 继续扩展这个能力，使模型可以点击、截图、测试自己的代码。MCP（Model Context Protocol）则把外部工具接入变成更标准的协议层，让 agent 不只在语言空间里推理，还能调用真实工具和服务。

工具使用与验证的互补关系

工具使用让 agent 能“做事”，验证让 agent 能“知道做得怎样”。只有前者，模型会盲目执行；只有后者，模型会停留在评论员角色。长时间运行需要两者叠加。

2.2 Claude Code research preview: 产品也是模型改进实验场

Claude Code 进入 research preview 时，讲者保留了一个很重要的发布意图：目标是更好地理解开发者如何把 Claude 用于 coding，从而反过来指导 future model improvements。换句话说，Claude Code 不是一个孤立工具，它也是模型改进的实验场。

这种定位解释了为什么 harness 和模型版本会一起演化。开发者在真实代码库中遇到的卡点，会暴露模型在 context、planning、verification 或 judgment 上的缺口；产品侧把这些缺口做成 SDK、工具、权限、上下文和回退机制；训练侧又可以从这些模式中吸收可泛化能力。讲者后续提到，随着模型变强，harness 的某些部分会变得没那么必要，或者换成新的形态。

在 May 附近的 Opus 4 和 Sonnet 4 阶段，讲者强调模型在管理自己的 context、走向 task completion、减少 reward hacking 等方面变好，Claude Code 同时进入 GA，并发布 Claude Code SDK。这里的 SDK 本质上是把 Claude Code 内部 harness 抽象成可复用的构建材料。

2.3 Ralph loop: 可预测失败优先于不可预测成功

Ralph loop 的直觉很朴素：把一个任务送进 Claude Code CLI，让它循环执行，直到任务完成。讲者提醒，这个技术经常被过度简化。更完整的 Ralph loop 包含几个阶段：先把提示拆成若干 features，再选择一个任务，开一个 fresh context window，然后在干净上下文中继续推进。

它之所以吸引人，是因为它体现了 *deterministically bad in an undeterministic world* 这个原则：在不确定世界里，可预测地失败，比不可预测地成功更适合工程迭代。可预测失败意味着工程师能复盘、调整停止条件、改进计划拆分和验证流程；不可预测成功则难以复制，也难以判断下一次为什么会坏。

讲者还指出 Claude Code 自己实现过一个 Ralph loop 变体。这个变体运行在单个 Claude Code session 内,不创建 fresh context window,而是依赖随时间发生的 compaction。它通过 max iterations、safe word 和 stop hook 控制循环:当 Claude 原本要停下时,stop hook 介入,如果 exit criteria 未满足,就继续运行。

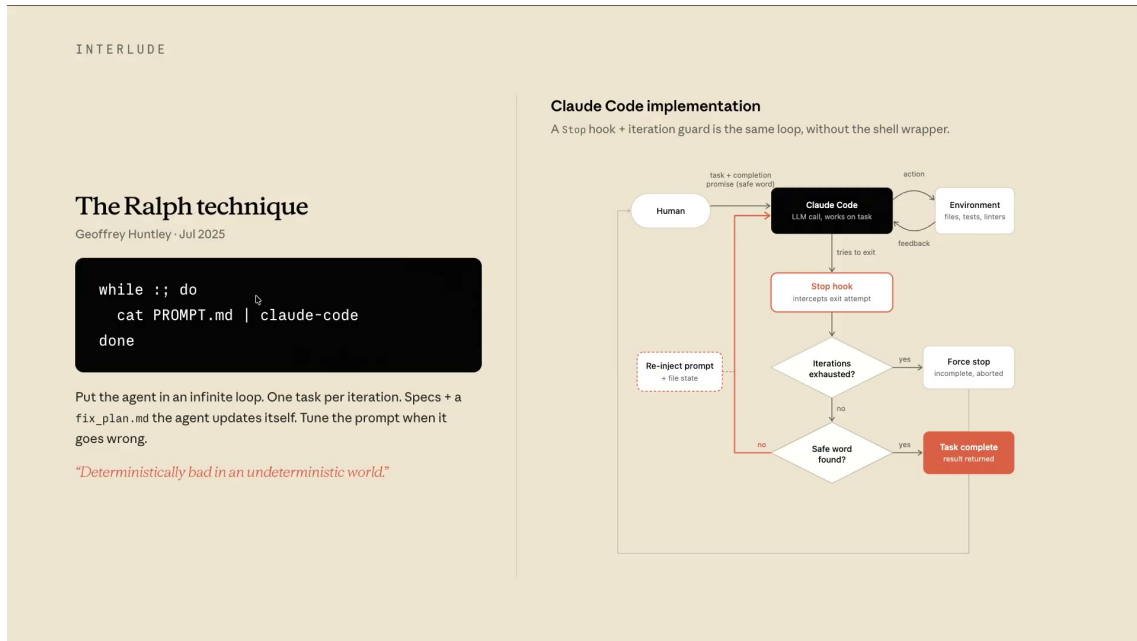


图 3: Ralph Loop 的核心不是机械重复,而是计划、循环、停止条件和上下文策略的组合。³

Ralph loop 不是机械重复

把 Ralph loop 理解成“重复运行同一个 prompt”会丢掉关键机制。真正有价值的是计划拆分、任务选择、fresh context 或 compaction 策略、停止条件和可复盘失败。

2.4 Checkpoints、Agent SDK 与技能的 progressive disclosure

Sonnet 4.5 阶段,模型开始更好地处理自己的 context,例如追踪已经消耗的 token,在接近上下文窗口末端时主动管理上下文。Claude Code 2.0 同时引入 checkpoints (检查点):记录代码随时间变化的状态,使会话可以 rewind 到较早位置。对于长时运行 agent,这相当于给施工现场拍阶段性快照;当后续步骤走偏时,系统能回到较可信的中间状态,而不是从一堆混乱修改里手工抢救。

同一阶段,Claude Code SDK 被重命名为 Agent SDK。这个命名变化说明 harness 不再只服务 coding。长时间运行的核心 primitives 可以迁移到其他领域:任务循环、工具权限、上下文加载、子代理、回退、状态文件和验收都不是 coding 独有问题。

随后 Haiku 4.5 与 Opus 4.5 补齐模型家族,使大量 sub-agents 变得更经济。Opus 4.5 更擅长 planning,因此可以承担规划角色;Sonnet 4.5 则作为 workhorse 执行代码。这种分工把前一章的 planning 问题具体化:规划可以交给更强规划模型,执行可以交给成本更合适的工作模型。

skills 的引入则直接处理 context 负担。讲者使用 progressive disclosure 这个术语:先只加载 skill 的 front matter,而不是把所有工具描述、引用材料和脚本一次性塞进上下文。只有当 skill 被真正实例化时,才加载正文、references,甚至可确定执行的代码。再加上 programmatic tool

³视频画面时间区间:00:08:01-00:08:28。若为裁剪图,时间区间仍对应原视频画面。

calling，模型可以临时写代码批量调用工具，只把最终结果送回上下文，避免大量原始工具输出污染 context。

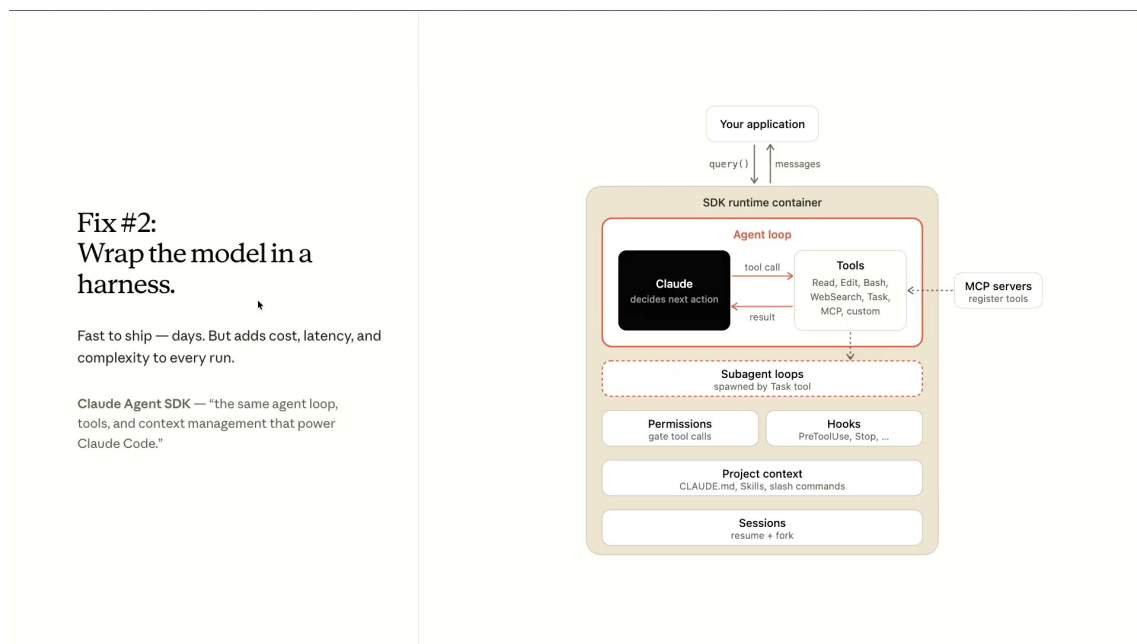


图 4: Agent SDK 把模型、工具、MCP、权限、Hooks、项目上下文和子 Agent 组织成可运行的 harness。⁴

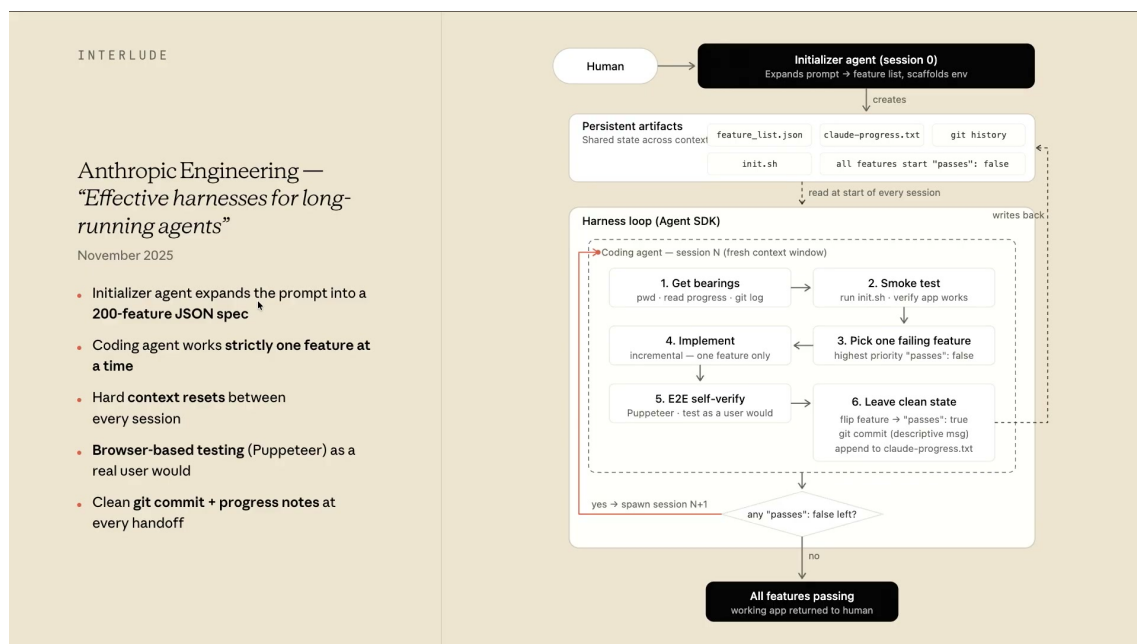


图 5: 第一代 long-running harness 用持久化工件、单功能推进、E2E 验证和 Git 状态维持连续性。⁵

⁴ 视频画面时间区间：00:05:00–00:05:27。若为裁剪图，时间区间仍对应原视频画面。

⁵ 视频画面时间区间：00:13:37–00:14:03。若为裁剪图，时间区间仍对应原视频画面。

progressive disclosure 的工程价值

Progressive disclosure 不是为了让提示词看起来整洁，而是为了控制上下文预算。长时运行中，最稀缺的资源不是工具数量，而是模型在关键时刻还能看见正确、少量、可执行的信息。

2.5 第一代 long-running harness: persistent artifacts + feature loop

到 November 左右，Anthropic 发布第一篇 long-running agents 博文。讲者用一个典型模糊需求说明第一代 harness：用户只说“write me a browser”“create a Slack clone”或“create a Salesforce clone”。这类提示太粗，不能直接让 agent 从头跑到尾；因此 harness 先由 initializer agent 把需求拆成 persistent artifacts（持久化产物）。

这些持久化产物包括：Featurelist.json，记录一组功能；progress file，记录进度；Git repo，承载代码状态；init script，固定启动方式；以及每个 feature 是否完成、是否通过测试的标志。讲者特别提到，他们发现模型可能覆盖 Markdown 文件，而较少覆盖 JSON 文件，所以 Featurelist.json 这样的结构化文件更适合做长期状态。

随后 harness 进入循环。每一轮都在 fresh context window 中重新取得方位：当前目录是什么，progress file 记录了什么，init script 怎样启动，smoke test 是否通过。然后只选择一个未完成 feature，完成实现，进入 verification loop，用 Puppeteer 像人类测试者一样实际验证行为。如果通过，就写 Git commit，并把该 feature 状态改为 pass；如果还有未完成 feature，就以新的 fresh context 继续下一轮。

文件系统状态为什么重要

Context 里的记忆会腐化，文件系统里的状态可以被后续 agent、人类和工具重新读取。第一代 harness 把“我做到哪了”从模型短期记忆迁移到 JSON、progress file、Git commit 和 init script，这就是长时运行可恢复性的起点。

2.6 Agent Teams 与 server-side compaction

Opus 4.6 与 Sonnet 4.6 阶段，设计空间再次变化。讲者称 Opus 4.6 是 very agentic model：它更擅长决定使用哪些工具，也能运行更久。前文 meter chart 中，从约 4 小时跳到 12 小时的变化，就发生在这个阶段的 very simple harness 口径下。Sonnet 4.6 则以更接近 Opus 的智能水平、更接近 Sonnet 的价格，成为 Claude Code 的 workhorse。

Agent Teams 的关键变化在协调结构。传统 sub-agent fan-out 中，所有子代理都回报给 main agent，main agent 成为瓶颈。Agent Teams 允许 sub-agents 相互沟通、协调，只在必要时向 main agent 汇报。这把长时运行从“一个主管问所有人”推进到“多个角色之间可以局部协作”的结构。

Server-side compaction 与 1M context GA 则改变上下文策略。过去的 Ralph loop 更强调 fresh context window；现在，模型上下文更大，服务端可以自动压缩，长会话可以在单一 session 中继续运行更久。这不意味着 fresh context 永远不需要，而是说明设计取舍会随模型版本改变：如果模型能在大上下文和 compaction 下保持足够 coherence，harness 可以减少某些重置成本；如果具体任务仍受 context rot 影响，fresh session 仍有位置。

Every release, agents run longer unattended

Beat	Model	Harness shipped with it	Primary attack	METR p50 horizon ¹	With harness ²
Feb '25	Sonnet 3.7	Claude Code	Planning	~1h (60 min)	"tasks that would normally take 45+ min, in a single pass"
May '25	Opus 4 / Sonnet 4	CC GA, Code SDK, code-exec, Files API	All three	~1h 40m (100 min)	Rakuten: 7h autonomous refactor
Sep '25	Sonnet 4.5	CC 2.0, checkpoints, memory tool, Agent SDK	Context	~1h 53m (113 min)	30+ h sustained focus on multi-step tasks
Nov '25	Opus 4.5 / Haiku 4.5	Skills, prog. tool calling, tool search, compaction	Context + Planning	~4h 53m (293 min)	Multi-day claude.ai clone build (200+ feature spec)
Feb '26	Opus 4.6 / Sonnet 4.6	Agent Teams, server compaction, 1M ctx	Verification	~12h (719 min)	6h retro game maker / 4h DAW, end-to-end

¹ METR-Horizon-v1.1, minimal scaffold, 50% success (corrected Mar 3 '26). ² Anecdotal, from Anthropic announcements & engineering blogs.

图 6: 模型发布、harness primitives 和无人值守时长共同演化。⁶

讲者最后把这一年压缩成一个工程判断：harness 不会因为模型变强而消失。更准确的说法是，harness 会随模型变强而演化。团队先找到模型缺口，用 harness 补上；再把这个 harness 行为反馈给模型训练或产品 primitive；当模型能力追上后，旧 scaffold 可能被删减，新的瓶颈又会出现。

Claude Code 演化的主线

这一年真正的主线不是“发布了多少版本”，而是三类失败被逐步工程化：context 由 skills、programmatic tool calling、server-side compaction 和 1M context 缓解；planning 由 Opus 规划、initializer agent、feature list 和 Agent Teams 缓解；judgment 由 verification loop、Puppeteer、checkpoints 和后续 evaluator 模式继续补强。

2.7 本章小结

本章从 Claude Code 的 prehistory 写到 Agent Teams 与 server-side compaction，核心答案是：长时运行 primitives 是模型能力与 harness 共同演化的结果。Sonnet 3.5、Computer Use 和 MCP 让模型能写、能看、能调用工具；Claude Code research preview 把真实开发使用变成模型改进信号；Ralph loop 把长任务变成可预测循环；checkpoints、Agent SDK、skills 和 programmatic tool calling 降低上下文与恢复成本；persistent artifacts 和 feature loop 把状态落到文件系统；Agent Teams 与 server-side compaction 又改变了协作和上下文设计空间。回到第一章的三类失败，同一组失败在不同模型阶段会需要不同 primitive。

3 当前 harness 结构：把 builder 和 judge 分开

本章的核心结论是：当前长时间运行 agent 的关键 harness 模式，是把 Generator/Builder 和 Evaluator/Judge 分离成不同角色、不同上下文和不同系统提示，让构建者负责产出，让评估者以更

⁶ 视频画面时间区间：00:15:47–00:16:14。若为裁剪图，时间区间仍对应原视频画面。

苛刻的标准操作真实环境并返回 critique。⁷ 这种结构解决的核心问题来自上一章的 judgment 缺口：模型容易把半成品看成完成品，也容易被自己的实现路径说服。角色分离给系统引入了 adversarial pressure（对抗式压力），再通过 explicit rubric（明确评分规约）把主观质量变成可反复优化的局部目标。

本章结论

本章先给出架构答案：Planner 设置方向，Generator 构建产物，Evaluator 使用 Playwright、rubric 和独立上下文来测试、评分、批评。这里的目标并非把“品味”变成全局客观真理；它是在一个声明清楚的本地标准下，让质量可以被检查、比较和爬坡。

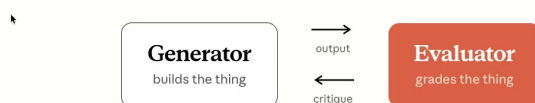
3.1 Frontier moves：为什么 harness 仍然存在

讲者用一句话概括模型和 harness 的关系：*the frontier doesn't really shrink, it just moves*。这句话的意思是，模型变强以后，旧的脚手架有些会被删掉，有些会被简化，可工程边界会移动到新的失败点。上一阶段需要新会话、显式任务循环或硬编码检查；下一阶段可能需要更强的评价器、更精细的验收契约，或者更好的 trace reading。

对长时间运行 agent 来说，harness（智能体运行框架）像工地上的脚手架：楼层低时，脚手架支撑基本施工；楼层变高后，脚手架的形态变了，安全绳、验收点、材料交接仍然存在。模型权重提升可以减少一部分外部控制，可只要任务持续数小时，系统仍要处理上下文、计划、验收、状态持久化和失败恢复。

这一章讨论的是 Anthropic 当时正在试验的一种简单但有效的模式：把构建和判断拆开。讲者明确说，这类模式既用于 fancy one-shot demo apps，也用于思考 post-training 和 RL 中如何让模型更适合 autonomous work。它服务展示效果，同时也是一种把失败模式工程化拆解的方式。

Separate the builder from the judge



Inspired by GANs. The evaluator uses Playwright to navigate the live page before scoring.

图 7: Generator 负责构建，Evaluator 负责批判；建造与判断被拆成两个角色。⁸

⁷本章对应视频源区间：00:17:28–00:25:04。

⁸视频画面时间区间：00:18:17–00:18:42。若为裁剪图，时间区间仍对应原视频画面。

3.2 GAN 类比: generator builds, evaluator grades

讲者把这个结构类比为 GAN (Generative Adversarial Network, 生成对抗网络): 一边是 generator, 另一边是 discriminator 或 evaluator, 两者之间形成 adversarial pressure。这个类比的教學价值在于说明“角色分离”和“压力方向”, 并不表示这个 harness 在数学上训练了一个 GAN。

在工程结构里, Generator 的职责是构建应用、页面、功能或代码; Evaluator 的职责是看产物是否真的成立。讲者的原话很紧凑: *generator builds, the evaluator grades*。这里的“grades”不只是读 diff, 也包括打开真实页面、点击、截图、尝试交互, 并把 critique 传回 Generator。也就是说, Evaluator 观察的是行为证据, 而不是只看代码形状。

GAN 类比的边界

GAN 类比可以这样理解: Generator 像参赛者, Evaluator 像评委。评委没有亲自做菜或画画, 却可以指出味道、构图、完整度和违和感。长时运行 agent 借用的正是这个能力差: 模型当 critic 往往比当 self-critical builder 更容易调校。

这个模式和常见的单 agent 循环差异很大。单 agent 循环通常让同一个 Claude Code 会话“做完以后检查自己”。这会把实现上下文、个人解释、局部妥协和验收判断混在一起。Generator/Evaluator 分离则让 Evaluator 拥有新的上下文和更窄的职责: 它不需要为实现辩护, 只需要判断产出是否满足标准。

3.3 为什么 evaluator 要独立

独立 Evaluator 的必要性来自一个具体偏差: 同一个模型在构建时会倾向于相信自己的中间解释。讲者指出, Evaluator 仍然是 LLM, 也仍然可能偏爱 LLM 风格的输出; 关键区别在于, *tuning a standalone critic to be harsh is actually very tractable*, 而让一个 builder 在构建过程中保持严格自我批评更难。

这可以用人类工作来类比: 一个人可以比较容易地批评一幅画、一顿饭或一个网页是否协调, 却未必能亲手画出同等水平的画、做出同等水平的饭、写出同等水平的前端。批评和生成使用相关能力, 但任务负担不同。Evaluator 的单一职责减少了认知负载, 也减少了“我已经做了这么多, 所以应该算完成”的自我合理化。

Tuning a standalone evaluator to be *skeptical* is tractable. Making a generator self-critical is not.

图 8: 独立 Evaluator 的关键洞见: 调出 skeptical critic 比调出 self-critical generator 更可控。⁹

Evaluator 独立不等于天然可靠

独立 Evaluator 仍然需要调校。讲者并没有说 LLM judge 天然可靠; 相反, 他明确承认 evaluator 会有 generosity bias 和 sycophancy bias。分离角色只是让“把 critic 调苛刻”这件事变得更可操作, 不能替代 rubric、真实交互和 trace review。

独立性还包括输入边界。Evaluator 最好主要评估输出和可观察行为, 而不是完整继承 Generator 的内部上下文。后续 Q&A 中讲者还会提到, 他们倾向使用 handoff/output judging, 因为把两个模型流的 thoughts 混在一起, 会污染 evaluator judgment。这一点和本章的角色分离是一致的。

3.4 Rubric: 把 subjective quality 写成可执行标准

主观质量的难点在于“好看”“有品味”“不俗套”这些判断很难直接写成单元测试。讲者的处理方式是: 如果团队对好坏有足够强的观点, 就把它写下来。这里的 rubric (评分规约) 承担 Evaluator 每轮评分和 critique 的标准来源角色。

视频中给出的四个高层 criteria 是:

- **Design quality:** 页面是否像一个 coherent whole, 整体是否一致。
- **Originality:** 决策是否有定制感, 还是只停留在 library defaults。
- **Craft:** typography、spacing、contrast 等技术执行是否到位。
- **Functionality:** 用户是否能在不猜测的情况下完成任务。

讲者还指出, 当时他们会把权重偏向 design 和 originality, 因为 Opus 4.6 在 functionality 上已经比较强, 真正想压制的是 purple gradients 和 generic AI slop 这类审美惯性。这里的权重属于随模型版本和任务目标调整的局部选择。

⁹视频画面时间区间: 00:19:46–00:20:12。若为裁剪图, 时间区间仍对应原视频画面。

Making subjective quality gradable

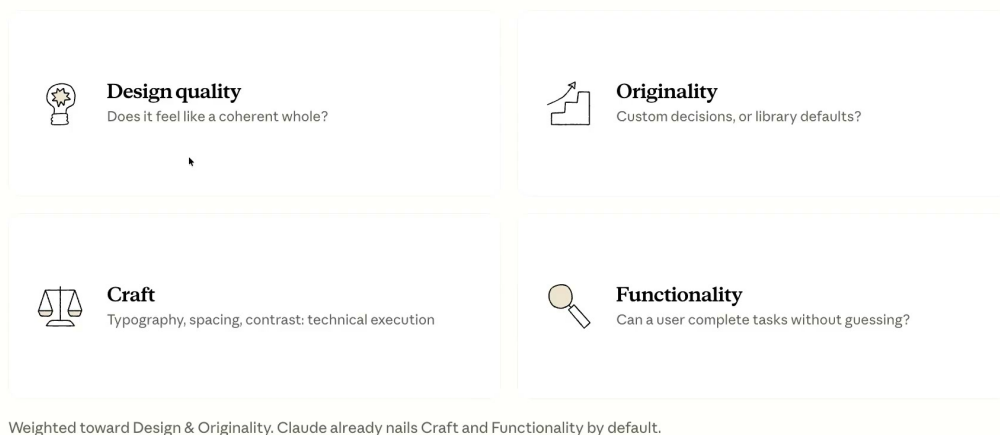


图 9: 主观质量被拆成 design quality、originality、craft、functionality 四个可评分维度。¹⁰

Rubric 的作用

Rubric 的作用是把“主观质量”变成“可执行的本地标准”。它不保证跨团队、跨产品、跨时代都一致；它保证当前 harness 能知道自己在爬哪座山。

校准 rubric 时，讲者提到会使用 few-shot reference sites，让 Evaluator 的 taste 向团队观点收敛。few-shot 的用途是提供正反例坐标：什么叫 coherent，什么叫 custom decision，什么叫 library defaults，什么叫 AI slop。

在运行循环里，Evaluator 会打开页面、导航、截图、按四项标准打分、写 critique，再交还给 Generator。这个机制还有一个单循环难以做到的能力：当 Generator 在某项标准上长期卡住，例如 originality 一直低分，GAN-style harness 可以丢弃当前方案并从头再试。长时运行的价值来自这种跨多轮的 course correction，而不是在同一个半成品上无限打补丁。

3.5 Planner：高层 PM 角色，而非细节技术计划器

当前端质量循环扩展到完整应用时，讲者加入了第三个角色：Planner。Planner 的职责是把一行 prompt 转成 deliberately high-level spec，并把 general workflow 拆成 sprints。它更接近 PM（产品经理）角色：定义方向、边界和阶段目标。

讲者特别强调 Planner 不应该提前规划过细的技术细节。原因很直接：模型仍然可能在技术细节上犯错；如果这个错误写进了上游 plan，它会在多个 sprint 中串联放大。对于数小时任务，早期错误就像施工图中的错误尺寸，后续每一层都会跟着偏。

¹⁰ 视频画面时间区间：00:21:18–00:21:43。若为裁剪图，时间区间仍对应原视频画面。

Scaling to full-stack: three agents



图 10: 三角色 full-stack harness: Planner 定方向, Generator 按 sprint 构建, Evaluator 用 rubric 与浏览器验证。¹¹

三角色组织类比

Planner、Generator、Evaluator 可以被理解为 PM、IC、QA 的简化组织结构。这个组织结构并不新,新的地方在于每个角色拥有自己的上下文窗口、职责边界和交接协议。

高层 Planner 的好处是保留探索空间。它告诉 Generator 和 Evaluator 本阶段要达成什么用户价值,却不提前锁死每个实现细节。随后更具体的 done criteria 会在 Generator 和 Evaluator 之间协商形成,这正是下一章的 contract-first 机制。

3.6 本章小结

本章建立了当前 harness 的核心结构: Planner 设置高层方向, Generator 构建产物, Evaluator 独立测试、评分和批评。它的底层假设是,长时间运行 agent 的质量瓶颈常常出现在 judgment,而不是单纯出现在生成能力。把 builder 和 judge 分开,可以引入 adversarial pressure,降低自评偏差,并让主观质量在显式 rubric 下局部可优化。

本章的限制也必须保留: Evaluator 仍然可能宽容, rubric 只在本地标准下有效, GAN 只是角色分离的类比, Planner 过度细化会制造级联错误。下一章会把这个三角色结构推进到更具体的工程协议:生成前先协商 what done actually means,再让 Evaluator 按契约操作真实应用。

4 Contract-first 生成: Retro Forge 案例与可测试验收

本章的核心结论是:长时间运行 agent 的质量提升来自 contract-first (先定义验收契约) 的工程流程。Generator 和 Evaluator 在写代码前先协商 *what done actually means*, 把用户故事转成可测试断言,再用 Playwright/browser validation 和 trace reading 持续修正评价标准。¹² Retro Forge

¹¹ 视频画面时间区间: 00:23:31-00:23:57。若为裁剪图,时间区间仍对应原视频画面。

¹² 本章对应视频源区间: 00:25:04-00:34:14。

案例显示，同一模型、同一简单 prompt，在 solo loop 和 full harness 下会产生明显不同的行为可靠性。

本章结论

Contract-first 的关键在于先把“完成”的含义写成可检查对象。Planner 给出高层方向，Generator 和 Evaluator 协商验收条件，Evaluator 按契约评分，初始的一句话需求只提供上游方向。

4.1 先定义 done，再写代码

讲者把 Generator 和 Evaluator 之间的“胶水”称为本节最有意思的部分：在 Generator 写下第一行代码之前，两个 agent 会先协商 done 的含义。Generator 可能提出：“我要构建 X feature，你应该通过 Y test 验证它。” Evaluator 可以反驳：“这个 scope 太大，测试太弱，还漏了 XYZ edge case。”

这一步把模糊的 user story 转成更具体的 testable assertions（可测试断言）。如果只有 Planner 在开头 one-shot 一个 spec，后续执行会受限於最初那份粗略计划。Contract negotiation 让验收标准在构建前变得可争论、可修改、可落盘。

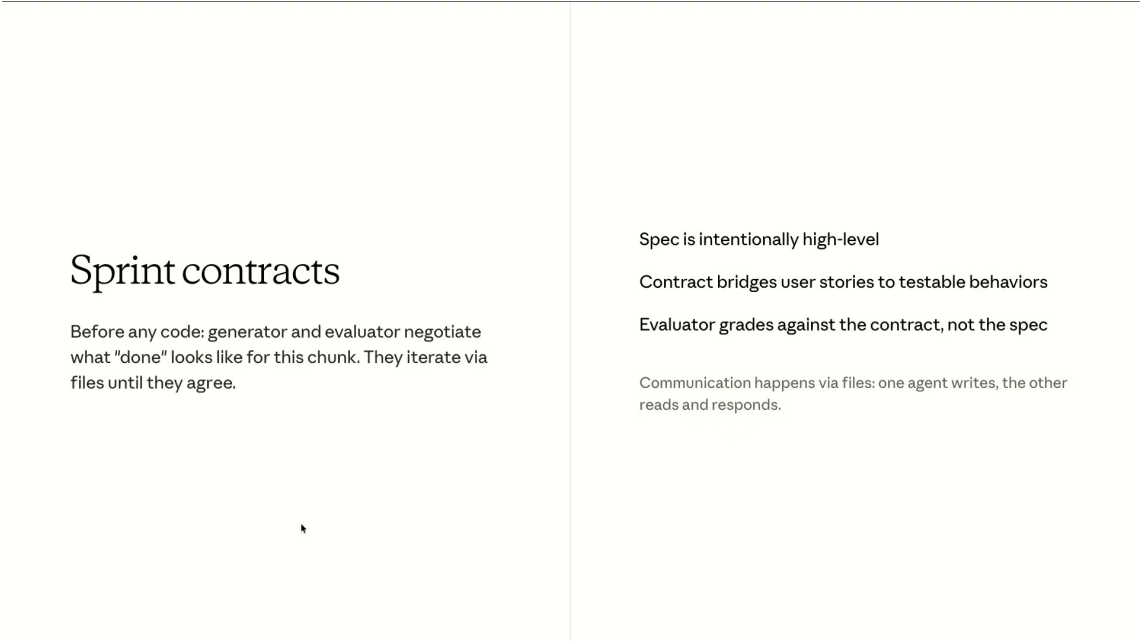


图 11: Sprint contract 把高层规格转成可测试行为，使 Evaluator 后续按 contract 评分。¹³

模糊 done 的风险

这里的“done”不能写成“页面看起来不错”或“功能基本可用”。模糊 done 会带来模糊 critique，Generator 会继续猜测。好的 done 应该包含行为、边界条件、测试方式和失败时可定位的证据。

Ralph loop 的早期形态往往有固定 plan.md，但缺少一个独立角色去和主循环争论验收条件。

¹³视频画面时间区间：00:25:15–00:25:43。若为裁剪图，时间区间仍对应原视频画面。

Contract-first 的改进点在这里：它让计划和验收之间出现一段协商层，既避免 Planner 过度规定技术细节，又避免 Generator 自己决定是否完成。

4.2 文件协议：via files on disk

讲者明确说，Generator 和 Evaluator 的来回协商是 *via files on disk*：一个 agent 写 markdown，另一个 agent 读取并回应，持续迭代到双方同意。文件承担 negotiation medium（协商媒介）和 durable state（持久状态）两种角色，价值高于普通日志。

文件协议的好处有三点。第一，它让状态跨上下文窗口存在，后续 agent 或人类可以接手。第二，它把口头式判断变成可审查证据，谁提出了哪个测试、谁反驳了哪个 edge case，都可以回看。第三，它减少了对单个上下文窗口记忆的依赖，特别适合多小时任务。

文件系统作为交接板

文件系统在这里相当于施工现场的交接板：Planner 写目标，Generator 写实现计划，Evaluator 写验收意见。下一班工人或审查者不需要猜上一轮发生了什么，只要读磁盘上的契约、进度和评价记录。

这个机制也解释了为什么后续章节会强调 JSON breadcrumbs、timestamped log 和 live docs。长时间运行 agent 的记忆不能只存在模型上下文里；它需要可以被别的 agent、工具和人类共同读取的外部状态。

4.3 Retro Game Maker 对照：solo loop 与 harness loop

讲者用同一个 prompt 做了对照实验：*build a retro game maker*。solo loop 的结果并非完全失败；它有 opening screen、sprite editor、canvas、palette、frame timeline 和 live preview，表面上“看起来像一个应用”。问题出在行为层：play mode 中有 entities、score、health，可箭头键和空格键没有实际效果。应用外观暗示功能完成，真实交互暴露了断裂。

full harness 版本则把产品命名为 Retro Forge，并由 Planner 做出更多产品层决策：new project dialog、nice canvas、54-color palette、8-bit preset、实际游戏比例的 sprite，以及 AI-level assistant。用户可以要求创建一个 castle with guards，harness 会把这类模糊 AI feature 转成具体工作项。

Example: retro game maker, solo vs full harness

	Solo agent	Full harness
Duration	20 minutes	6 hours
Cost	\$9	\$200
Play mode works?	No (entity wiring broken)	Yes
Sprite editor	Basic	Full palette, zoom, AI-assisted
Features in spec	4 (from prompt)	16 across 10 sprints

图 12: Retro Game Maker 对照显示：完整 harness 成本更高、耗时更长，但交付能力显著提升。¹⁴

这个例子中的成本和时间需要谨慎理解。视频中提到 full harness 约 ~\$200、6 小时；这是讲者展示实验时的具体例子，不是推荐预算，也不是所有任务都应采用的默认成本。它只说明同一模型在不同 scaffold 下可以进入不同的产品完成度。

Retro Forge 的真正差异

Retro Forge 的差异不应被理解为“模型突然更聪明”。讲者强调，差异 entirely just scaffolding：Planner 把模糊需求变成产品方向，Generator 构建，Evaluator 操作应用并按契约发现行为失败。

最关键的对照是 play mode。full harness 中 debug HUD 的数字是 live 的，physics loop 在运行，arrow keys 能移动 player，player 会和 castle walls 发生 collision。这些细节属于 Evaluator 真正操作应用后必须满足的行为证据，不能被当作 UI 装饰。

4.4 Evaluator 真正操作应用

Evaluator 的价值在 Retro Forge 案例中非常具体：它不是只检查代码是否编译，也不是只读取静态 diff。它会 launch the game、try to play it，并判断哪些功能需要被测试，才能证明“这个游戏真的能玩”。

行为级验证能发现 CI 或粗略 loop 放过的问题。讲者举到的例子包括 FastAPI route ordering：单元测试可能全过，但生产路径会因为 route 匹配顺序出错；delete key 的逻辑也可能因为 Boolean condition 错误导致功能失效。类似问题只有在 Evaluator 使用应用、点击、拖动、按键、观察状态变化时才容易暴露。

这给读者一个实用判断：如果任务产物是 Web app、native app、交互式工具或游戏，静态检查只覆盖了一部分质量。Evaluator 应该尽量观察用户可见行为，例如打开页面、点击按钮、拖拽画布、触发快捷键、读取 HUD、检查状态是否同步。

¹⁴ 视频画面时间区间：00:27:01–00:27:27。若为裁剪图，时间区间仍对应原视频画面。

4.5 27 条 contract criteria 与可行动 critique

讲者说，Retro Forge 这个应用最终有 27 条 contract criteria。这个粒度不是偶然的。对于长时间运行 harness，criteria 太少会让 critique 变成泛泛而谈；criteria 足够具体时，Generator 才知道应该修哪一行、哪个 handler、哪个行为。

视频中的可行动失败包括：

- rectangle fill 只在 drag start/end 放置 tile，fillRectangle 存在但没有在 mouseup 触发；
- delete key 需要同时满足 selection 和 selected entity id，实际应允许其中一个条件成立；
- PUT /frames/reorder 被定义在 /{frame_id} 之后，FastAPI 把 reorder 当成 frame id，从而返回 422。

What the evaluator actually caught

Contract criterion	Evaluator finding
Rectangle fill: click-drag fills area	FAIL. Only places tiles at drag start/end. fillRectangle exists but isn't triggered on mouseUp.
Delete key removes placed entities	FAIL. Handler requires both selection AND selectedEntityId. Clicking only sets one. Condition should be OR.
Reorder animation frames via API	FAIL. PUT /frames/reorder defined after /{frame_id}. FastAPI matches 'reorder' as a frame_id int, returns 422.

图 13: Evaluator 捕获的失败具体到交互、状态和 API 路由，critique 因此可以直接转成修复。¹⁵

Contract criteria 没有固定条数

“27 条”不是固定配方。它表达的是验收标准需要细到能驱动修复。对于小功能，可能 5 条足够；对于多小时应用，20 多条也可能只是起点。真正的判断标准是 critique 是否能指向具体行为或具体代码位置。

这里也能看出 contract-first 与普通测试驱动的互补关系。单元测试适合稳定接口和纯逻辑；contract criteria 还要覆盖用户行为、视觉一致性、交互路径、边界条件和产品意图。它更像“验收测试 + 设计 rubric + 真实浏览器操作”的组合。

4.6 Trace reading: 调 prompt 像读 stack trace

讲者对 Evaluator 的局限说得很直接：Claude out of the box 是一个很弱的 general QA agent。原因是 judge 系统里常见的 sycophancy 和 generosity bias 也会出现在这里。早期运行中，QA agent

¹⁵ 视频画面时间区间：00:31:16–00:31:42。若为裁剪图，时间区间仍对应原视频画面。

可能发现一个 bug，然后说 “later fix it”，接着就结束。这说明即使有独立 Evaluator，也需要调校其标准和行为。

调校的方法没有神秘捷径。讲者说，真正的 art 是 reading the traces：读 agent 实际做了什么，找出它的 judgment 和人类判断分歧的位置，再把这个分歧反馈进 prompt。这个过程像读 stack trace：沿着执行轨迹定位判断失真点，再决定是否需要更多实验。

The debugging loop

Read the evaluator's logs. Find
where its judgment diverges from
yours. Update the prompt. Repeat.

图 14: 调试 harness 的循环：读 Evaluator 日志，找到判断分歧，更新 prompt，再重复。¹⁶

Trace reading 的目标

Trace reading 的目标是校准 Evaluator 的判断边界。若 Evaluator 太宽容，contract 再细也会被放过；若 Evaluator 只给笼统评价，Generator 无法修复。高质量 harness 需要同时打磨 contract、工具操作和 evaluator prompt。

讲者还给了一个工具提示：把 agent transcripts pipe 到文件里，用 grep 或另一个 agent 做第一轮定位，让另一个 agent play through traces，并更新 prompt。这种做法把长 trace 的初筛自动化；真正关键的分歧点仍需要人类或高质量 reviewer 理解。

4.7 本章小结

本章把上一章的三角色结构落到可执行流程：先协商 *what done actually means*，通过 *via files on disk* 把契约持久化，再让 Evaluator 操作真实应用并按 granular contract criteria 给出 critique。Retro Forge 的案例说明，长时间运行质量来自 scaffold、contract、browser validation 和 trace tuning 的组合。

本章也保留了两个风险边界。第一，full harness 的实验成本和运行时间很高，不能被当成默认推荐预算。第二，产物可以显著强于 solo loop，但视频没有声称它达到 production-ready。下一章将继续讨论：当模型版本变强后，哪些 harness 部件可以瘦身，哪些评价纪律仍然需要保留。

¹⁶ 视频画面时间区间：00:33:17–00:33:42。若为裁剪图，时间区间仍对应原视频画面。

5 模型变强后，harness 怎样瘦身

本章对应视频 00:34:14--00:40:05。核心结论是：harness（智能体运行框架）应当随着模型能力更新而删减，旧模型需要的 context reset、严格 sprint decomposition、频繁 evaluator cadence，在新模型上可能变成额外成本；可长期保留的部分是职责分离、可执行评估和可追踪状态。

5.1 Spiky behaviors：先找模型缺口

讲者在这一段先澄清一个常见误判：模型变强以后，harness design 并没有失去意义。更准确的判断方式是先观察每个模型的 **spiky behaviors**，也就是某个模型在真实长任务中反复出现的尖刺型弱点，然后让 harness 填补这些缺口。

这里的 **spiky behaviors** 可以理解为“平时看起来很强、到了某类情境突然掉链子”的行为。例如一个模型可以写出很好的单个功能，却在两小时任务中忘记最初目标；可以通过单轮测试，却在 UI 细节上过早自信；可以理解文件结构，却在长上下文后开始急于收尾。harness 的设计对象正是这些可复现的局部缺口。

harness 的版本意识

好的 harness 先回答“当前模型在哪些环节会稳定失败”，再决定要加哪些 scaffold。模型版本变化后，原先用于弥补缺口的 scaffold 需要重新接受评估。

这也是本 workshop 的一条主线：模型权重和 harness 共同演化。模型侧把某些能力内化以后，harness 侧就应当把相应的外部约束降级、合并或删除。否则系统会把旧时代的补丁继续当成新任务的默认架构。

5.2 Opus 4.5 到 4.6：harness on a diet

视频中最具体的例子是 Opus 4.5 到 Opus 4.6 的变化。讲者用一个很形象的说法描述这个过程：**harness on a diet**，即让运行框架“减脂”。它保留 generator-evaluator 结构的核心职责，同时删掉那些已经不再产生收益的循环细节。